

brew67potions

Proof of Concept (POC) & Product Development Document (PDD)

Version 1.0 | September 2026 | Implementation Baseline

Implementation baseline for product, design, engineering, operations, and validation

Document Control

Document	POC & Product Development Document
Product	brew67potions
Version	1.0
Status	Draft implementation baseline

Contents

1. Executive Summary
2. Product Vision and Objectives
3. Problem Statement
4. Scope and Release Strategy
5. Hypotheses and Validation Plan
6. User Personas and Primary Journeys
7. Product Catalog
8. Functional Requirements
9. Non-Functional Requirements
10. Technical Architecture
11. Logical Architecture Flow
12. Data Model
13. API Contract Outline
14. Recommendation Engine Design
15. Sustainability and Impact Measurement
16. Security, Privacy, and Compliance
17. Analytics and Observability
18. Testing Strategy
19. Deployment and Operations
20. Delivery Roadmap
21. Acceptance Criteria
22. Key Risks and Mitigations
23. Definition of Done
24. Immediate Next Actions

1. Executive Summary

brew67potions is a digital commerce and education platform that presents sustainable home, garden, and personal-care products as collectible 'potions' sourced from three narrative biomes: Forest, Ocean, and Mountain.

The product combines a story-led shopping experience with concentrated, refill-oriented product formats. The flagship product, Brew No. 67 (The Tri-Realm Catalyst), is positioned as a universal multi-surface concentrate kit containing Forest, Ocean, and Mountain-inspired ingredients.

The implementation will begin with a measurable proof of concept and then evolve into a production e-commerce platform. The system must support biome discovery, product exploration, a guided potion configurator, impact estimates, customer accounts, inventory, checkout, subscriptions, and an administrative operating layer.

Product safety, regulatory compliance, formulation validation, and environmental claims are release gates. The software must not present unverified formulation, biodegradability, carbon, plastic-saving, or health-related claims as established facts.

2. Product Vision and Objectives

Vision: Make sustainable purchasing feel understandable, engaging, and repeatable by combining practical refill products with an immersive biome-based apothecary experience.

Primary objectives:

- Validate whether the narrative shopping experience increases qualified engagement and purchase intent.
- Validate whether customers understand concentrated/refill product formats and their preparation instructions.
- Establish a trustworthy foundation for product, ingredient, inventory, order, and impact data.
- Build an architecture that can move from simulated checkout to real payments without rewriting the core domain model.
- Provide measurable product analytics for acquisition, engagement, configurator completion, conversion, retention, and refill behavior.

3. Problem Statement

Conventional sustainable products often require customers to compare unfamiliar materials, dilution ratios, packaging claims, and environmental tradeoffs without a clear decision path. A standard catalog can make the experience feel transactional and difficult to understand.

brew67potions addresses this through three connected experiences: biome-led discovery, guided product configuration, and transparent impact explanations. The platform must preserve practical clarity underneath the narrative layer so customers know exactly what they are buying, how to use it, and what claims are supported.

4. Scope and Release Strategy

POC scope:

- Responsive landing page with Forest, Ocean, and Mountain biome portals.
- Product catalog with biome filtering and product detail pages.
- Brew No. 67 configurator using explicit, versioned recommendation rules.
- Simulated cart and checkout intent capture.
- Analytics events for page views, biome selection, configurator steps, CTA clicks, and email/pre-order submission.
- Basic administrative seed data and product/formulation management.

Production scope:

- Customer authentication and account management.

- PostgreSQL-backed catalog, formulations, inventory, orders, payments, refunds, subscriptions, and fulfillment status.
- Stripe or Shopify payment integration after legal, tax, product, and operational readiness checks.
- Customer-facing impact dashboard with traceable calculation sources.
- Admin console, audit logs, role-based access control, monitoring, backups, and incident procedures.

Out of scope for the first production release:

- Automated scientific proof of environmental claims without validated source data.
- Medical, therapeutic, or ingestible product claims.
- Fully autonomous formulation generation.
- Multi-country tax, shipping, and regulatory support unless explicitly added to the release plan.

5. Hypotheses and Validation Plan

H1: A biome-based narrative selector improves qualified engagement compared with a conventional product-category entry point.

H2: A guided configurator increases understanding of the recommended product format and reduces uncertainty before checkout.

H3: Concentrated/refill formats can reduce packaging and shipping burden, but the actual percentage must be calculated from verified packaging, product, and logistics data.

H4: A coherent cross-biome product story can improve product discovery and basket composition.

Validation approach:

- Run a baseline landing page and a narrative landing page where practical.
- Track unique visitors, qualified CTA clicks, configurator starts, configurator completions, email submissions, checkout starts, and purchase conversions.
- Use surveys or short interviews to measure comprehension, trust, perceived value, and purchase barriers.
- Record experiment version, audience, traffic source, and sample size for every reported metric.
- Avoid declaring a causal lift from a small or uncontrolled sample. Report observed differences with confidence intervals or clear limitations.

6. User Personas and Primary Journeys

Primary customer: A consumer seeking lower-waste home products who wants simple instructions, understandable ingredients, and credible environmental information.

Secondary customer: A sustainability-oriented household or small business buyer seeking repeatable refill purchasing and predictable product performance.

Internal operator: A product or operations administrator who manages products, formulations, inventory, claims evidence, orders, and customer support.

Core journeys:

1. Discover: Visitor lands on the site, explores a biome, views products, and learns the product format.
2. Configure: Customer answers usage questions, receives a versioned recommendation, reviews assumptions, and adds a product to the cart.
3. Purchase: Customer reviews product details, shipping information, policies, and total price, then completes checkout.
4. Refill: Customer sees refill compatibility, usage guidance, and reorder options from their account.
5. Operate: Admin updates catalog data, inventory, formulation versions, claim evidence, and order state while retaining an audit trail.

7. Product Catalog

Forest Realm: Canopy and soil care. Initial concept product: Brew No. 14, Pine & Mycelium Grounding Drops. The final ingredient list, use case, safety profile, and claims require formulation and regulatory validation.

Ocean Realm: Marine and waterway safety. Initial concept product: Brew No. 45, Tidal Dissolving Pods. Claims about biodegradability, microplastics, and aquatic safety must be substantiated before public display.

Mountain Realm: High-altitude mineral care. Initial concept product: Brew No. 61, Glacial Mineral Elixir. Any water-treatment or household-use claims require product-specific testing and appropriate regulatory review.

Flagship product: Brew No. 67, The Tri-Realm Catalyst. Initial concept form factor: three dissolvable tablets plus a reusable glass spray bottle. The production formulation, compatibility, stability, storage, and usage instructions must be finalized before sale.

Every product record must include: name, brew number, biome, product type, form factor, unit size, price, currency, ingredients, usage instructions, warnings, storage instructions, evidence-backed claims, availability status, and version history.

8. Functional Requirements

FR-01 Biome exploration: The system shall display Forest, Ocean, and Mountain portals with accessible navigation, descriptions, imagery, and product links.

FR-02 Catalog filtering: Users shall filter products by biome, product type, availability, and refill compatibility.

FR-03 Product detail: Each product page shall display price, size, usage instructions, ingredients, warnings, packaging information, shipping details, and supported claims.

FR-04 Potion configurator: Users shall provide water hardness, household size, primary usage area, and other validated inputs. The system shall return a recommendation with a rule-set version and explanation.

FR-05 Recommendation transparency: The configurator shall show the inputs used, assumptions, limitations, and the reason for the recommendation. It shall not imply scientific precision beyond validated rules.

FR-06 Cart: Users shall add, remove, and modify eligible products and see subtotal, shipping estimate, taxes when available, and total.

FR-07 Intent capture: POC users shall submit email and consent preferences for pre-order or product-interest follow-up.

FR-08 Checkout: Production users shall be redirected to a trusted payment flow or an approved embedded payment experience. Payment data shall not be stored directly by the application.

FR-09 Inventory: The system shall track sellable inventory, reserved inventory, available inventory, and low-stock thresholds.

FR-10 Orders: The system shall persist order status, payment status, fulfillment status, refunds, and relevant timestamps.

FR-11 Impact estimates: The system shall calculate and display only metrics backed by documented assumptions and source data. Estimates must be labeled as estimates.

FR-12 Accounts: Customers shall be able to view orders, saved preferences, refill-compatible products, and subscription status when subscriptions are enabled.

FR-13 Administration: Authorized staff shall manage catalog, formulation versions, evidence records, inventory, and order operations.

FR-14 Auditability: Material changes to prices, claims, formulations, inventory adjustments, and order state shall be recorded.

9. Non-Functional Requirements

Performance: Target Lighthouse performance score of 90 or higher for core pages under a defined test environment. Target first-load experience should be measured with realistic mobile and desktop profiles.

Accessibility: Target WCAG 2.1 AA for navigation, forms, focus states, contrast, keyboard operation, error messaging, and dynamic configurator updates.

Security: Enforce secure session handling, server-side authorization, input validation, rate limiting, secure headers, secrets management, dependency scanning, and least-privilege database access.

Privacy: Collect only necessary personal data, document retention periods, provide consent controls where required, and avoid sending unnecessary personal data to analytics tools.

Reliability: Production services should have health checks, structured logs, error monitoring, backups, and a documented recovery process.

Maintainability: Use typed domain models, API contracts, migrations, automated tests, code review, and environment-specific configuration.

Digital sustainability: Optimize images, JavaScript bundles, caching, server-side rendering, and third-party scripts. The 1.5 MB bundle target is a budget to validate rather than a guarantee across every route.

10. Technical Architecture

Recommended baseline: Next.js App Router with TypeScript for the web application, a typed backend API using NestJS or FastAPI, PostgreSQL for transactional data, object storage for product media, and a managed payment provider.

Frontend responsibilities: routing, server-rendered catalog pages where appropriate, accessible interactive components, cart state, configurator UI, analytics instrumentation, and authenticated account views.

Backend responsibilities: authentication, authorization, catalog queries, recommendation rules, pricing validation, inventory reservations, order orchestration, payment webhooks, impact calculations, and admin operations.

Database responsibilities: durable product and commerce records, versioned formulations, source-backed claims, inventory movements, customer records, and audit events.

External services: payment provider, email provider, analytics platform, error monitoring, and optional shipping/tax integrations. Each integration must have a clear failure mode and retry strategy.

Recommended deployment separation: web frontend, API service, worker/background jobs, PostgreSQL, and object storage. The initial POC may use a simplified deployment, but domain boundaries should remain explicit.

11. Logical Architecture Flow

Browser → Next.js web application → backend API → PostgreSQL.

Backend API → payment provider for checkout session creation and webhook verification.

Backend API → email provider for transactional messages and consented marketing workflows.

Backend API → analytics/event pipeline for product and funnel events, with privacy controls.

Worker → asynchronous tasks such as email delivery, inventory reconciliation, webhook retries, and impact report generation.

Admin console → authenticated backend endpoints with role-based access control and audit logging.

12. Data Model

Core entities:

- **Biome:** biome_id, name, description, display_order, active_status, conservation_partner_reference.
- **Product:** product_id, brew_number, name, slug, biome_id, description, form_factor, price, currency, refillable, active_status, version.
- **Ingredient:** ingredient_id, name, source_biome_id, supplier_reference, upcycled_source, evidence_reference, status.
- **ProductIngredient:** product_id, ingredient_id, percentage or quantity, unit, formulation_version_id.
- **FormulationVersion:** version_id, product_id, version_label, status, effective_date, approved_by, safety_document_reference.

- ClaimEvidence: evidence_id, claim_text, evidence_type, source_reference, review_date, status, reviewer.
- Customer: customer_id, email, consent_status, created_at, updated_at.
- Cart and CartItem: cart_id, customer_id or anonymous_token, product_id, quantity, price_snapshot.
- Order and OrderItem: order_id, customer_id, totals, currency, payment_status, fulfillment_status, timestamps, line-item snapshots.
- InventoryItem and InventoryMovement: SKU, location, on_hand, reserved, adjustment_reason, actor, timestamp.
- RecommendationRuleSet: rule_set_id, version, inputs, conditions, output, explanation, active_status.
- ImpactFactor: factor_id, metric_type, factor_value, unit, source_reference, valid_from, valid_to.
- AuditEvent: actor, action, entity_type, entity_id, before_snapshot, after_snapshot, timestamp.

Use foreign keys, check constraints, indexes on slugs and status fields, unique constraints for brew numbers and SKUs, and migrations for every schema change.

13. API Contract Outline

GET /api/v1/biomes — list active biomes.

GET /api/v1/products — list products with filtering and pagination.

GET /api/v1/products/{slug} — retrieve a product detail view.

POST /api/v1/recommendations — evaluate a versioned configurator rule set.

POST /api/v1/carts — create or recover a cart.

POST /api/v1/carts/{id}/items — add an eligible item after server-side price and availability checks.

PATCH /api/v1/carts/{id}/items/{itemId} — update quantity.

POST /api/v1/checkout/session — create a payment-provider checkout session.

POST /api/v1/webhooks/payment — verify and process payment events idempotently.

GET /api/v1/orders — list the authenticated customer's orders.

GET /api/v1/orders/{id} — retrieve an authorized order.

POST /api/v1/intent-signups — capture POC interest with consent metadata.

Admin endpoints must be separated by authorization scope and must not trust client-provided prices, roles, inventory counts, or payment status.

14. Recommendation Engine Design

The Brew No. 67 configurator should initially use deterministic, testable rules rather than an opaque generative model.

Inputs may include household size, primary usage area, water hardness, desired purchase type, and product availability. Inputs must be validated against a controlled schema.

Output must include recommended SKU or kit, quantity, rationale, rule-set version, and warnings where the input is unsupported or insufficient.

Rules must be stored and versioned. A recommendation generated for a customer should retain the rule-set version so that later changes do not alter historical explanations.

Example rule behavior: if an input combination is unsupported, return a safe fallback recommendation or ask the user to choose a supported use case. Do not infer a formulation change solely from user preference without approved product rules.

15. Sustainability and Impact Measurement

Impact metrics must be based on a documented methodology. The initial concept's 'one tablet equals one plastic bottle saved' and '85% shipping reduction' statements should remain provisional until the comparison unit, baseline, system boundary, and evidence are defined.

Required impact data: baseline product packaging mass, refill packaging mass, product mass, shipment mass, transport assumptions, packaging composition, end-of-life assumptions, and source documentation.

Impact calculations should return: metric name, value, unit, baseline, comparison scenario, methodology version, source reference, and confidence or qualification text.

Display language should distinguish between measured results, modeled estimates, customer-specific calculations, and future targets.

Environmental partner information should be represented as a relationship and evidence record rather than a marketing claim without documentation.

16. Security, Privacy, and Compliance

Authentication: use secure, short-lived sessions or tokens, secure cookies where applicable, CSRF protection for cookie-based state changes, and account recovery controls.

Authorization: implement role-based access control for customer, support, operations, and administrator roles. Enforce authorization at the API layer.

Payments: use hosted checkout or tokenized payment-provider flows. Never store raw card data.

Webhooks: verify signatures, use idempotency keys, persist event IDs, and handle retries safely.

Data protection: encrypt data in transit, restrict production access, store secrets in a secret manager, redact sensitive fields from logs, and define deletion/retention policies.

Compliance readiness: obtain legal review for product labeling, consumer safety, environmental marketing claims, privacy notices, terms, refunds, subscriptions, and applicable tax obligations before production launch.

17. Analytics and Observability

Funnel events: `page_view`, `biome_viewed`, `product_viewed`, `configurator_started`, `configurator_step_completed`, `recommendation_generated`, `add_to_cart`, `checkout_started`, `intent_submitted`, `purchase_completed`, `refill_viewed`, and `subscription_started`.

Every event should include an event name, timestamp, anonymous or authorized user identifier, session identifier, page or product context, experiment version when applicable, and schema version.

Operational telemetry: request latency, error rate, database latency, queue depth, webhook failures, payment failures, inventory mismatches, and background job retries.

Dashboards: acquisition funnel, configurator funnel, conversion, product performance, inventory health, payment reliability, and application health.

Do not use analytics identifiers or event payloads that reveal unnecessary personal, payment, or sensitive information.

18. Testing Strategy

Unit tests: recommendation rules, pricing calculations, impact calculations, validation, permissions, and state transitions.

Integration tests: database repositories, checkout session creation, webhook verification, inventory reservation, and order creation.

End-to-end tests: biome discovery, product browsing, configurator completion, cart changes, checkout handoff, and authenticated order history.

Accessibility tests: keyboard navigation, focus management, semantic labels, screen-reader announcements, contrast, and error states.

Security tests: authorization bypass attempts, input validation, rate limits, secret exposure checks, dependency scanning, and webhook replay handling.

Performance tests: core route loading, API latency, database query plans, and realistic cart or catalog traffic.

Release gates: no critical security findings, passing migrations, successful backup verification, tested rollback procedure, and validated product claims for any claims displayed publicly.

19. Deployment and Operations

Environments: local development, test/CI, staging, and production. Use separate credentials and databases for each environment.

CI pipeline: formatting, linting, type checking, unit tests, integration tests, build verification, dependency/security checks, and migration validation.

Deployment: immutable builds, environment-specific configuration, health checks, controlled database migrations, and rollback instructions.

Backups: scheduled encrypted backups, retention policy, restoration drills, and documented recovery point and recovery time objectives.

Incident response: severity definitions, alert routing, customer communication procedure, incident timeline, root-cause analysis, and follow-up actions.

Admin operations: all manual inventory, price, formulation, claim, and order changes should be logged with actor and reason.

20. Delivery Roadmap

Phase 0 — Product and risk definition: confirm product categories, target customers, approved use cases, claims inventory, regulatory review needs, and measurement methodology.

Phase 1 — POC, weeks 1–4: build responsive landing page, biome selector, catalog mock data, Brew No. 67 configurator, simulated cart, intent capture, analytics, and usability testing.

Phase 2 — Formulation and sourcing, weeks 5–12: validate manufacturers, ingredient specifications, packaging, safety documentation, testing, supplier evidence, and operational costs.

Phase 3 — Commerce platform, weeks 13–18: implement production catalog, accounts, PostgreSQL, inventory, checkout, payment webhooks, order workflows, email, and admin controls.

Phase 4 — Beta launch, week 19 onward: launch a controlled founder or beta cohort, monitor fulfillment and support, validate repeat purchase behavior, and refine product and platform workflows.

Each phase requires an explicit exit review. A calendar estimate is not a launch commitment until product, legal, manufacturing, and operational dependencies are confirmed.

21. Acceptance Criteria

POC acceptance:

- A new user can explore all three biomes on mobile and desktop.
- Product cards and detail views render from structured data.
- A user can complete the configurator and receive a deterministic, explainable recommendation.
- A user can add a product to a simulated cart and submit interest with consent controls.
- Analytics events are emitted with a documented schema and can be validated in a test environment.
- Accessibility and performance checks are documented with known exceptions.

Production acceptance:

- Prices and inventory are validated server-side.
- Payment webhooks are verified and idempotent.
- Orders cannot be accessed across customer accounts.
- Formulation and claim records have versioning and approval status.
- Backup restoration is tested.
- Monitoring and alerting are operational.
- Product labels, usage instructions, claims, refund terms, privacy notices, and operational workflows are approved for launch.

22. Key Risks and Mitigations

Product safety risk: require formulation testing, safety documentation, clear instructions, and release approval before public sale.

Environmental claim risk: maintain claim evidence, methodology versions, review dates, and qualified display language.

Manufacturing risk: validate supplier capacity, minimum order quantities, quality controls, lead times, and batch traceability.

Customer comprehension risk: test the configurator and product instructions with users unfamiliar with concentrated formats.

Inventory risk: use reservations, idempotent order workflows, reconciliation jobs, and low-stock alerts.

Payment risk: rely on a certified provider flow, verify webhooks, and implement reconciliation.

Platform complexity risk: ship a narrow vertical slice first, keep recommendation logic deterministic, and defer nonessential integrations.

Brand trust risk: never make sustainability or performance claims that cannot be supported by current evidence.

23. Definition of Done

A feature is complete when its requirements are documented, acceptance tests pass, accessibility considerations are addressed, telemetry is added where appropriate, security implications are reviewed, error states are implemented, and deployment or rollback behavior is understood.

A release is complete when product, engineering, operations, and compliance stakeholders have approved the applicable release gates and the system has a documented support and incident path.

24. Immediate Next Actions

1. Freeze the initial POC feature list and define the target user segment.
2. Create the product and claim evidence inventory, separating confirmed facts from hypotheses.
3. Decide the backend framework: NestJS or FastAPI, based on implementation ownership and team familiarity.
4. Create the monorepo or repository structure for web, API, shared types, database migrations, and infrastructure.
5. Implement the first vertical slice: biome page → product detail → configurator → simulated cart → intent capture.
6. Define analytics event schemas before implementing the UI.
7. Create a risk register for formulation safety, packaging, claims, privacy, payments, and manufacturing.
8. Establish staging deployment, CI checks, and a minimal observability baseline before adding production commerce.

Appendix A: Initial Architecture Decisions

Decision Area	Initial Direction	Reason / Constraint
Frontend	Next.js + TypeScript	Responsive commerce, server rendering, typed UI flows.
Backend	NestJS or FastAPI	Select one before implementation; preserve a versioned API boundary.
Database	PostgreSQL	Transactional catalog, commerce, inventory, and audit requirements.
Payments	Hosted provider flow	Reduces direct payment-data handling and compliance burden.
Recommendations	Deterministic versioned rules	Explainable, testable, safer for v1.
Impact metrics	Source-backed calculation service	Separates measured evidence from marketing assumptions.

Appendix B: Open Decisions

- Which initial product is legally and operationally ready for pilot sale?
- Which backend framework will be used for the first production implementation?
- Will commerce be custom-built around a payment provider or delegated to Shopify?
- Which environmental claims have sufficient evidence for public display?
- What are the initial shipping regions, tax requirements, and fulfillment model?
- Which subscription and refill workflows are required for the beta release?
- What are the baseline products and system boundaries for impact comparisons?